

DAT12-2

Programming in Python

Automation, logic, and computation for data work

Write a script that reads data, processes it, and writes the result — versioned, debuggable, and reproducible.

Albert School — 40 hours

Preface

This is your first programming course, and it assumes nothing. You do not need to have written a line of code before. What you do need is a willingness to be precise: a computer does exactly what you tell it, no more and no less, and most of learning to program is learning to say what you mean without ambiguity.

Python is our instrument. It is one of the most widely used languages in the world, and the dominant language of data and artificial intelligence — the pillar this course belongs to. We chose it because it reads almost like English, because it hides machinery you do not yet need to see, and because nearly every tool you will meet later in your studies speaks Python.

The course has one destination, and every chapter is a step toward it: by the end you will write a Python script that reads data from a file, transforms it, and writes a new file — and you will be able to run it again tomorrow, on another machine, and get the same answer. That last property, **reproducibility**, is what separates a throwaway snippet from a tool. We build toward it deliberately: first the environment, then the language, then files, then the libraries and ideas that let your scripts reach beyond your own computer.

Read with your fingers on the keyboard. Type every example, change it, break it, and read the error message. Programming is not a spectator sport, and the error messages — which beginners learn to fear — are in fact the friendliest, most patient teacher you will have.

Contents

Preface	2
1 The development environment	3
1.1 A first script from the command line	3
1.2 The editor: VS Code	3
1.3 Virtual environments and <code>pip</code>	3
2 Core data types	4
2.1 Numbers: <code>int</code> and <code>float</code>	4
2.2 Text: <code>str</code>	4
2.3 The booleans: <code>bool</code>	5
2.4 Collections: <code>list</code> , <code>tuple</code> , <code>dict</code>	5
3 Control flow	6
3.1 Conditionals: <code>if</code> / <code>elif</code> / <code>else</code>	6
3.2 Loops: <code>for</code> and <code>while</code>	6
3.3 Iterating over a dictionary	7
4 Functions	7
4.1 Defining and calling	7
4.2 Typed parameters and docstrings	7
4.3 Scope	7
5 File input and output	8
5.1 Reading and writing with context managers	8
5.2 Handling errors with <code>try</code> / <code>except</code>	8
6 Debugging	9
6.1 Read the error message	9
6.2 Print and reason about state	9
7 Third-party libraries	10
7.1 Installing and importing	10
7.2 Reading documentation	10

8	Virtual environments, revisited	10
8.1	What an environment isolates	10
8.2	Reproducing from <code>requirements.txt</code>	10
9	Object-oriented programming	11
9.1	Defining a class, creating an object	11
9.2	Reading the machinery	11
10	Data processing: from CSV to CSV	12
10.1	The shape of a CSV file	12
10.2	A complete processing script	12
11	A first encounter with APIs	13
11.1	HTTP GET and the <code>requests</code> library	13
11.2	Parsing a JSON response	13

1 The development environment

Before writing Python you must be able to run it. A program is just text in a file; what turns that text into behaviour is the **Python interpreter**, a program already installed (or installable) on your machine that reads your file line by line and carries out its instructions.

1.1 A first script from the command line

Create a file called `hello.py` containing one line:

```
print("Hello, world!")
```

Open a terminal in the folder that holds the file and type:

```
python hello.py
```

The terminal prints `Hello, world!`. That is the whole loop you will repeat thousands of times: edit a file, run it from the command line, read the output. The word `python` here names the interpreter; `hello.py` is the **argument** you hand it — the file to execute.

Remark On some systems the interpreter is called `python3` rather than `python`, because an older Python 2 still lingers under the name `python`. If `python` reports a version starting with 2, use `python3` everywhere in this book.

1.2 The editor: VS Code

You **could** write Python in any text editor, but a code editor earns its keep by understanding the language. We use **Visual Studio Code** (VS Code): it colours your code so structure is visible at a glance, underlines mistakes before you run anything, and has an integrated terminal so you never leave the window to run your script. Install VS Code, then install its official **Python extension** — that extension is what brings the language-aware features to life.

1.3 Virtual environments and `pip`

Real scripts use code other people wrote — **packages**. You install packages with `pip`, Python's package installer:

```
pip install requests
```

But where does that package go? If `pip` installed everything into one shared location, two projects needing two different versions of the same package would collide. The fix is a **virtual**

environment: a private, project-local folder holding its own copy of the interpreter and its own packages.

```
python -m venv .venv          # create an environment in the folder .venv
source .venv/bin/activate     # activate it (macOS / Linux)
.venv\Scripts\activate       # activate it (Windows)
```

Once activated, your prompt usually shows `(.venv)`, and any `pip install` now writes into that folder only. We return to virtual environments in depth in Section 8, where they become the key to reproducibility; for now simply know that **every project gets its own environment**, created right after you create the project.

Common pitfall Forgetting to activate the environment is the most common first-week confusion. If `import requests` fails with `ModuleNotFoundError` even though you just installed it, check that `(.venv)` appears in your prompt — you almost certainly installed it into one environment and ran your script in another.

2 Core data types

Every value in Python has a **type**, and the type determines what you can do with the value: you can divide numbers but not words, you can capitalise words but not numbers. Getting the type right is the first thing to get right.

2.1 Numbers: `int` and `float`

There are two kinds of number. An **int** is a whole number — 0, 42, -7 — with unlimited size. A **float** is a number with a decimal point — 3.14, -0.5, 2.0 — stored in a fixed amount of memory, which has a consequence we will flag shortly.

```
apples = 12          # int
price = 2.50         # float
total = apples * price # 30.0 -> a float, because price is a float
```

Notice that mixing an int and a float yields a float. Division always yields a float, even when it divides evenly:

```
10 / 2    # 5.0 (float) -- "true" division
10 // 3   # 3 (int) -- "floor" division, drops the remainder
10 % 3    # 1 (int) -- "modulo", the remainder
```

Common pitfall Floats cannot represent every decimal exactly. Typing `0.1 + 0.2` gives `0.30000000000000004`, not `0.3`. This is not a Python bug; it is how fractional numbers are stored in binary on every computer. The practical rule: **never test two floats for exact equality**. Instead of `if x == 0.3`, ask whether they are close enough, e.g. `if abs(x - 0.3) < 1e-9`.

2.2 Text: `str`

A **string** is a sequence of characters, written between quotes — single or double, your choice, as long as they match:

```
name = "Ada"
greeting = 'Hello, ' + name + '!' # 'Hello, Ada!' -- "+" joins strings
```

Strings are full of useful built-in operations called **methods**, reached with a dot:

```
" Ada ".strip()      # 'Ada'      -- remove surrounding spaces
"ada".upper()        # 'ADA'
"a,b,c".split(",")   # ['a', 'b', 'c'] -- a list (see below)
```

The cleanest way to build a string from values is an **f-string** — a string prefixed with **f**, where anything in **{}** is replaced by its value:

```
apples = 12
print(f"I have {apples} apples.") # I have 12 apples.
print(f"Each costs {2.5:.2f} euros.") # Each costs 2.50 euros. (: .2f -> 2 decimals)
```

2.3 The booleans: **bool**

A **bool** is one of exactly two values, **True** or **False**. Booleans are the answers to yes/no questions, and they are what conditionals (next chapter) act on:

```
12 > 10      # True
"a" == "b"   # False
3 == 3.0     # True -- equal in value, even across int and float
```

2.4 Collections: **list**, **tuple**, **dict**

Three types hold **many** values at once, and choosing between them is one of the recurring judgement calls of Python.

A **list** is an ordered, changeable sequence, written with square brackets. Reach an element by its **index**, counting from zero:

```
scores = [90, 85, 72]
scores[0]      # 90 -- first element
scores[-1]     # 72 -- last element
scores.append(64) # scores is now [90, 85, 72, 64]
scores[1] = 88  # change in place -> [90, 88, 72, 64]
```

A **tuple** is an ordered sequence that *cannot* change once created, written with parentheses. Use it for a fixed group of values that belong together — a coordinate, an RGB colour — where changing one without the others would be a mistake:

```
point = (3, 4)
point[0]      # 3
point[0] = 9   # TypeError: tuples cannot be changed
```

A **dict** (dictionary) stores **key** → **value** pairs and looks things up by key rather than by position. It is the right type whenever your data is “labelled”:

```
student = {"name": "Ada", "age": 19}
student["name"]      # 'Ada' -- look up by key
student["age"] = 20   # change a value
student["city"] = "Paris" # add a new pair
```

Type	Ordered?	Changeable?	Use it for
list	yes	yes	a sequence you grow or edit
tuple	yes	no	a fixed group that belongs together
dict	by key	yes	data looked up by a label
str	yes	no	text

Table 1: Choosing a collection type. “Changeable” is the word programmers shorten to *mutable*.

Remark You can ask any value its type with `type(x)`, and convert between types with the type's name: `int("42")` turns the string "42" into the number 42, and `str(42)` does the reverse. Reading a number from a file always gives you a string first — converting it is a step beginners routinely forget.

3 Control flow

So far our programs run straight down, top to bottom. **Control flow** lets a program choose what to do and repeat work — the difference between a list of instructions and an actual algorithm.

3.1 Conditionals: `if` / `elif` / `else`

A conditional runs a block of code only when a condition is `True`. Picture grading a score:

```
score = 78
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
print(grade)          # C
```

Python checks each condition in order and runs the **first** block whose condition is true, skipping the rest. `elif` (“else if”) chains alternatives; the final `else` catches everything left over.

Common pitfall Two pitfalls live in this small example. First, **indentation is not decoration in Python — it is the syntax**. The indented lines are what “belongs to” the `if`; get the indentation wrong and the meaning changes or the program refuses to run. Use four spaces per level and be consistent. Second, do not confuse `=` with `==`: a single `=` **assigns** a value, while `==` **compares**. `if score = 90` is an error; you mean `if score == 90`.

3.2 Loops: `for` and `while`

A **for loop** repeats once for each item in a collection. This is the loop you will reach for nine times out of ten, because most work is “do something to every item”:

```
scores = [90, 85, 72]
total = 0
for s in scores:
    total = total + s
print(total)          # 247
```

To repeat a fixed number of times, loop over `range(n)`, which yields the numbers 0, 1, ..., n-1:

```
for i in range(3):
    print(i)           # prints 0, then 1, then 2
```

A **while loop** repeats as long as a condition stays true. Use it when you do not know in advance how many repetitions you need:

```
balance = 100
years = 0
```

```
while balance < 200:
    balance = balance * 1.05 # 5% growth per year
    years = years + 1
print(years) # 15
```

Common pitfall A while loop whose condition never becomes false runs forever — an **infinite loop**. Make sure something inside the loop changes a value the condition depends on. If a program hangs, press Ctrl+C in the terminal to stop it, then look for the variable your condition tests and check that the loop body actually moves it.

3.3 Iterating over a dictionary

Dictionaries are iterable too. Looping over a dict gives its keys; `.items()` gives key and value together — a pattern you will use constantly when processing data:

```
prices = {"apple": 2.5, "pear": 3.0}
for fruit, price in prices.items():
    print(f"{fruit} costs {price} euros")
```

4 Functions

As soon as you find yourself writing the same lines twice, you want a **function**: a named, reusable block of code that takes inputs and (usually) returns an output. Functions are how programs are kept small enough to understand — each one a box you can reason about without looking inside the others.

4.1 Defining and calling

Suppose we keep converting Celsius to Fahrenheit. We package the rule once:

```
def to_fahrenheit(celsius):
    return celsius * 9 / 5 + 32

print(to_fahrenheit(100)) # 212.0
print(to_fahrenheit(0)) # 32.0
```

`def` introduces a function; `to_fahrenheit` is its name; `celsius` is a *parameter*, a placeholder filled in when you **call** the function. `return` hands a value back to whoever called it. A function without `return` hands back the special value `None`.

4.2 Typed parameters and docstrings

Python lets you annotate what types a function expects and returns. These **type hints** do not change how the code runs, but they document intent and let your editor catch mistakes:

```
def to_fahrenheit(celsius: float) -> float:
    """Convert a temperature from Celsius to Fahrenheit."""
    return celsius * 9 / 5 + 32
```

The triple-quoted string just under the `def` line is a **docstring**: a sentence saying what the function does. It is not a comment for you alone — tools display it as the function’s official help, and `help(to_fahrenheit)` will print it. Write one for every function.

4.3 Scope

A variable created inside a function exists only inside it — its **scope** is local to that call. This is a feature, not a limitation: it means a function’s inner workings cannot accidentally clobber a variable somewhere else.

```
def square(x):
    result = x * x      # 'result' lives only inside square
    return result

print(square(5))        # 25
print(result)           # NameError: 'result' is not defined out here
```

Common pitfall The reverse direction is a classic trap. A function **can read** a variable defined outside it, so a function that forgets to take a value as a parameter may appear to work by silently using an outside variable — until that outside variable changes and the function’s behaviour changes with it. Pass what a function needs as parameters; do not rely on values reaching in from the surrounding code.

5 File input and output

A script that only prints to the screen forgets everything when it ends. To do real work it must read files and write files — to **persist** data.

5.1 Reading and writing with context managers

Opening a file gives you a connection to it that must eventually be closed, or you risk losing data and leaking resources. The clean way is the `with` statement — a **context manager** that closes the file automatically when the block ends, even if something goes wrong inside:

```
# Writing
with open("notes.txt", "w") as f:    # "w" = write (replaces the file)
    f.write("first line\n")          # \n is a newline
    f.write("second line\n")

# Reading
with open("notes.txt", "r") as f:    # "r" = read (the default)
    content = f.read()               # whole file as one string
print(content)
```

The variable `f` is the open file, valid only inside the `with` block. To read a file line by line — the usual case for data — loop over it directly:

```
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())          # .strip() removes the trailing newline
```

Common pitfall The mode matters. Opening with `"w"` **erases** the file before writing — a great way to destroy data you meant to keep. Use `"a"` (append) to add to the end of an existing file, and `"r"` to read without any risk of changing it.

5.2 Handling errors with `try / except`

What if the file is not there? Or a line you expected to be a number is actually the word “unknown”? An **exception** is Python’s way of signalling that something went wrong; left unhandled, it stops the program. The `try / except` block lets you catch the problem and respond instead of crashing:

```
try:
    with open("data.csv", "r") as f:
        content = f.read()
```



```
except FileNotFoundError:
    print("No data file found; starting empty.")
    content = ""
```

Python **tries** the first block; if the named exception occurs, it runs the **except** block instead. You can catch different problems differently:

```
try:
    value = int(line)
except ValueError:
    print(f"Skipping line that is not a number: {line!r}")
```

Common pitfall Resist the urge to write a bare **except:** that swallows **every** error. It hides the bugs you most need to see — a typo in a variable name raises an exception too, and a blanket **except** will quietly bury it. Catch the specific exception you expect, and let the unexpected ones surface.

6 Debugging

Your programs will not work the first time. None of anyone's do. **Debugging** — finding and fixing the gap between what you told the computer and what you meant — is not a sign of failure; it is the main activity of programming. The good news is that it is a learnable, systematic skill, not a knack.

6.1 Read the error message

When Python hits an error it prints a **traceback**: a report ending with the error type and message, and pointing at the line where it happened. Beginners skim past it in a panic; experts read it carefully, from the bottom up. The last line is usually the most informative:

```
Traceback (most recent call last):
  File "script.py", line 7, in <module>
    total = total + scores[5]
                        ~~~~~^^^
```

```
IndexError: list index out of range
```

This tells you **what** (an `IndexError`), **where** (line 7), and **why** (a list index out of range — you asked for `scores[5]` in a list that does not have six items). Three of the four questions answered before you have thought at all. Memorising the common error types — `NameError`, `TypeError`, `ValueError`, `IndexError`, `KeyError`, `FileNotFoundError` — turns most tracebacks into instant diagnoses.

6.2 Print and reason about state

When the message alone is not enough, the oldest tool still works: print the values your code depends on, just before the line that misbehaves, and check whether they are what you assumed.

```
for i, line in enumerate(lines):
    print(f"line {i}: {line!r}")    # what is REALLY in each line?
    value = int(line)
```

Programming is reasoning about **program state** — the current values of all your variables. A bug is almost always a place where the real state differs from the state you imagined. Printing makes the real state visible; the moment you see it, the bug usually explains itself.

Remark The `!r` inside an f-string prints the **repr** of a value — its programmer-facing form, with quotes and escape characters shown. `print(line)` and `print(f"{line!r}")` look different for a string like `"42\n"`: the second reveals the hidden newline that the first hides, and that newline is exactly the kind of thing that breaks `int(line)`.

7 Third-party libraries

Python’s true power is its **ecosystem**: hundreds of thousands of packages written by others, each a problem already solved. Learning to find, install, and use them is as important as the language itself.

7.1 Installing and importing

We met `pip` already. Inside an activated virtual environment, install a package once:

```
pip install requests
```

Then **import** it in your script to make its contents available:

```
import requests          # use it as requests.get(...)
from datetime import date # bring one name out of a package
import pandas as pd      # import under a shorter alias
```

7.2 Reading documentation

You cannot — and need not — memorise a library. The skill is reading its **documentation**: the official description of what each function expects and returns. When you meet a new library, find its docs, look at the first example, copy it, run it, then adapt it to your need. Every library worth using has a “Quickstart” page written for exactly this moment. Learning to read docs well is the single habit that most separates a self-sufficient programmer from a stuck one.

8 Virtual environments, revisited

We created a virtual environment back in the first chapter; now we can say **why** it matters, because we finally have packages to isolate.

8.1 What an environment isolates

A virtual environment isolates **which packages, at which versions** a project sees. Project A can use version 1 of a library while project B uses version 2, with no conflict, because each has its own `.venv` folder. It does not isolate your files or your data — only the installed packages and the interpreter.

8.2 Reproducing from `requirements.txt`

Here is where isolation pays off. Inside an activated environment, freeze the exact versions you are using into a file:

```
pip freeze > requirements.txt
```

The file lists every package and its exact version:

```
certifi==2024.2.2
charset-normalizer==3.3.2
requests==2.31.0
```

Anyone — a teammate, a grader, or **you** in six months on a new laptop — can now recreate your environment exactly:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt # -r = read packages from this file
```

This is the heart of **reproducibility**, the property this whole course builds toward. A script plus its `requirements.txt` is a promise: run these on a fresh machine and you will get the same result I did. Commit both to version control; never commit the `.venv` folder itself — it is large, machine-specific, and fully rebuildable from the requirements file.

9 Object-oriented programming

So far our data and the functions acting on it have been separate. **Object-oriented programming** (OOP) bundles them together: a **class** is a blueprint that packages data (*attributes*) with the functions that act on it (*methods*). You have used objects all along — a string's `.upper()` is a method, a list's `.append()` is a method. Now you will define your own.

9.1 Defining a class, creating an object

Imagine tracking bank accounts. Each account has an owner and a balance, and supports deposits and withdrawals. Rather than juggling loose variables, we define a class:

```
class Account:
    def __init__(self, owner: str, balance: float = 0.0):
        self.owner = owner # an attribute
        self.balance = balance # an attribute

    def deposit(self, amount: float) -> None:
        """Add money to the account."""
        self.balance = self.balance + amount

    def withdraw(self, amount: float) -> None:
        """Remove money, refusing to overdraw."""
        if amount > self.balance:
            raise ValueError("insufficient funds")
        self.balance = self.balance - amount
```

To create an **object** (an *instance*) from the class, call the class like a function:

```
acc = Account("Ada", 100.0) # runs __init__
acc.deposit(50) # acc.balance is now 150.0
acc.withdraw(30) # acc.balance is now 120.0
print(acc.owner, acc.balance) # Ada 120.0
```

9.2 Reading the machinery

Two pieces of syntax deserve a careful look.

`__init__` is the **constructor**: a special method Python runs automatically when you create an object, used to set up its initial attributes. The default `balance: float = 0.0` means the argument is optional — `Account("Bo")` starts at zero.

`self` is the object itself. Every method takes `self` as its first parameter, and through it reaches the object's own attributes: `self.balance` inside a method is “this particular account's balance”. When you write `acc.deposit(50)`, Python quietly passes `acc` as `self`.

Common pitfall Two `self` mistakes are near-universal at first. Forgetting `self` in a method's parameter list (`def deposit(amount):`) makes Python complain it got an unex-

pected argument. And forgetting `self`. when using an attribute (`balance = balance + amount`) creates a throwaway local variable instead of changing the object — the deposit silently does nothing. If a method “runs but changes nothing”, a missing `self`. is the first suspect.

10 Data processing: from CSV to CSV

We now assemble everything into the script the whole course points at: read a data file, process it, write a new one. The standard format for tabular data is **CSV** — comma-separated values, a plain-text table where each line is a row and commas separate the columns.

10.1 The shape of a CSV file

Suppose `students.csv` contains:

```
name,score
Ada,90
Bo,72
Cy,55
```

The first line is a **header** naming the columns. Python’s built-in `csv` module reads such a file, and `csv.DictReader` hands each row back as a dictionary keyed by the header — exactly the labelled data a dict is made for:

```
import csv

with open("students.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row)      # {'name': 'Ada', 'score': '90'}
```

Common pitfall Look closely: `row["score"]` is the **string** `'90'`, not the number `90` — everything read from a CSV arrives as text. Compute with it and you must convert first: `int(row["score"])`. Forgetting this gives the baffling `"90" + "72" = "9072"` instead of `162`, because `+` on strings joins them.

10.2 A complete processing script

Here is the destination, in full: read the students, keep only those who passed, and write them to a new file with a computed column. It uses data types, a loop, a conditional, type conversion, file I/O, and error handling — the whole course in twenty lines.

```
import csv

PASS_MARK = 60

def load_students(path: str) -> list:
    """Read a CSV of students into a list of dicts, scores as ints."""
    students = []
    with open(path, "r") as f:
        for row in csv.DictReader(f):
            try:
                row["score"] = int(row["score"])
            except ValueError:
                print(f"Skipping bad row: {row!r}")
```

```

        continue          # skip to the next row
    students.append(row)
return students

def write_passers(students: list, path: str) -> None:
    """Write students who passed, with a 'result' column, to a new CSV."""
    with open(path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["name", "score", "result"])
        writer.writeheader()
        for s in students:
            if s["score"] >= PASS_MARK:
                s["result"] = "PASS"
                writer.writerow(s)

students = load_students("students.csv")
write_passers(students, "passers.csv")
print(f"Processed {len(students)} students.")

```

Run it and `passers.csv` appears, holding Ada and Bo with a new `result` column, Cy dropped. Notice how the work split naturally into two small, named functions — one to read, one to write — each testable on its own. That structure is not decoration; it is what makes a script you can debug and trust.

11 A first encounter with APIs

The last reach beyond your own machine: getting data from the internet on demand. An **API** (Application Programming Interface) is a defined way for one program to ask another for data or services. Web APIs let your script request data from a server — weather, exchange rates, anything — and get a structured reply.

11.1 HTTP GET and the `requests` library

The web runs on **HTTP**, a protocol of requests and responses. The simplest request is a **GET**: “please give me the data at this address (URL).” The `requests` library makes a GET in one line:

```

import requests

response = requests.get("https://api.github.com/users/python")
print(response.status_code)    # 200 means success

```

The **status code** is the server’s verdict: 200 means OK, 404 means not found, 500 means the server itself erred. Always check it before trusting the reply.

11.2 Parsing a JSON response

Web APIs almost always answer in **JSON** — a text format for nested data that maps cleanly onto Python’s dicts and lists. `requests` parses it for you with `.json()`:

```

import requests

response = requests.get("https://api.github.com/users/python")
if response.status_code == 200:
    data = response.json()          # a dict, parsed from JSON
    print(data["name"])             # 'Python'
    print(data["public_repos"])    # an int
else:
    print(f"Request failed: {response.status_code}")

```

Once `.json()` hands you a dict, you are back on familiar ground: reach into it by key exactly as in Table 1’s dictionary, loop over lists of results, pull out the fields you need. An API call is, in the end, just one more source of the labelled data you already know how to process — and a natural place to plug into the CSV pipeline of the previous chapter: fetch with `requests`, parse with `.json()`, write with `csv`.

Common pitfall Network requests fail in ways file reads do not — the server may be down, the connection may drop, the address may be wrong. Wrap a real request in `try / except` (catching `requests.exceptions.RequestException`) and always check `status_code`; a script that assumes the internet always answers will crash the first time a train goes through a tunnel.

Where you have arrived. You can set up a project with its own virtual environment, write and run a script from the command line, wield Python’s core types and control flow, factor logic into documented functions, read and write files safely, debug by reading tracebacks and inspecting state, install and use third-party libraries, model data with your own classes, and — the whole point — read a CSV, process it, and write a new file, with the exact recipe captured in a `requirements.txt` so anyone can reproduce your result. That is a real programmer’s toolkit. Everything later in the Data & AI pillar is built on it.